

JAVA

From Fundamentals to Professional Mastery

A Complete Guide for Beginners to Advanced Professionals

Lokesh Belvo

Table of Contents

Chapter 1: Introduction to Java	3
1.1 What is Java?	3
1.2 Key Features of Java	3
1.3 JDK, JRE, and JVM Explained	3
1.4 Setting Up Your Development Environment	4
1.5 Your First Java Program	4
Chapter 2: Java Fundamentals — Syntax, Variables, and Data Types.....	6
2.1 Structure of a Java Program	6
2.2 Variables and Identifiers	6
2.3 Primitive Data Types	6
2.4 Reference Types and the String Class	6
2.5 Type Casting and Conversion.....	7
2.6 Operators	7
Chapter 3: Control Flow — Decisions and Loops	8
3.1 Conditional Statements	8
3.2 The switch Statement and Switch Expressions	8
3.3 Loops	8
3.4 break, continue, and Labeled Loops	9
Chapter 4: Arrays and Strings	10
4.1 Arrays	10
4.2 Multidimensional Arrays.....	10
4.3 The Arrays Utility Class	10
4.4 Deep Dive: The String Class	10
4.5 Common String Methods.....	11
4.6 StringBuilder and StringBuffer	11
Chapter 5: Object-Oriented Programming — Classes and Objects	12
5.1 Classes and Objects.....	12
5.2 Constructors.....	12
5.3 Access Modifiers and Encapsulation	12
5.4 Static vs Instance Members	13
5.5 The Object Class and Method Overriding.....	13
Chapter 6: Object-Oriented Programming — Inheritance, Polymorphism, and Abstraction	15
6.1 Inheritance	15
6.2 Method Overriding and super	15
6.3 Polymorphism	15
6.4 Abstraction: Abstract Classes and Interfaces	16
6.5 Composition Over Inheritance.....	17

Chapter 7: Exception Handling	18
7.1 What Are Exceptions?	18
7.2 The Exception Hierarchy	18
7.3 try-catch-finally	18
7.4 try-with-resources	18
7.5 Custom Exceptions	19
7.6 Best Practices	19
Chapter 8: The Java Collections Framework	20
8.1 Overview	20
8.2 List	20
8.3 Set	20
8.4 Map	20
8.5 Queue and Deque	21
8.6 Choosing the Right Collection	21
8.7 Sorting with Comparable and Comparator	21
Chapter 9: Generics	22
9.1 Why Generics?	22
9.2 Generic Classes	22
9.3 Generic Methods	22
9.4 Bounded Type Parameters and Wildcards	22
Chapter 10: Multithreading and Concurrency	24
10.1 Threads and Processes	24
10.2 The Executor Framework	24
10.3 Synchronization	24
10.4 Locks and Atomic Variables	25
10.5 CompletableFuture	25
10.6 Common Pitfalls	25
Chapter 11: File I/O and Serialization	26
11.1 Streams: Byte vs Character	26
11.2 Reading and Writing Text Files	26
11.3 The Modern NIO.2 API (java.nio.file)	26
11.4 Object Serialization	26
Chapter 12: Modern Java — Lambdas, Streams, and Optional	28
12.1 Lambda Expressions	28
12.2 Functional Interfaces	28
12.3 The Stream API	28
12.4 Common Stream Operations	29
12.5 Optional	29
12.6 Records (Java 16+)	29
Chapter 13: Database Connectivity with JDBC	30
13.1 What is JDBC?	30
13.2 Core JDBC Workflow	30

13.3 PreparedStatement vs Statement	30
13.4 Transactions	30
13.5 Connection Pooling	31
Chapter 14: Design Patterns in Java	32
14.1 Why Design Patterns Matter	32
14.2 Singleton Pattern	32
14.3 Factory Pattern	32
14.4 Builder Pattern	32
14.5 Observer Pattern	33
14.6 Strategy Pattern	33
14.7 Dependency Injection	34
Chapter 15: Java Memory Management and Garbage Collection	35
15.1 The JVM Memory Model	35
15.2 Garbage Collection Fundamentals	35
15.3 Garbage Collector Algorithms	35
15.4 Memory Leaks in Java	35
15.5 Tuning Tips	35
Chapter 16: Annotations and Reflection	36
16.1 Built-in Annotations	36
16.2 Custom Annotations	36
16.3 Reflection	36
Chapter 17: Networking in Java	37
17.1 Sockets	37
17.2 HTTP Clients	37
17.3 RESTful Web Services	37
Chapter 18: Testing Java Applications with JUnit	38
18.1 Why Automated Testing Matters	38
18.2 Writing a Basic Test	38
18.3 Lifecycle Annotations	38
18.4 Mocking with Mockito	38
18.5 The Testing Pyramid	39
Chapter 19: Build Tools and Professional Best Practices	40
19.1 Maven	40
19.2 Gradle	40
19.3 Code Style and Readability	40
19.4 Version Control Discipline	40
19.5 Logging	41
Chapter 20: Introduction to the Spring Framework	42
20.1 Why Spring?	42
20.2 The IoC Container and Beans	42
20.3 Building a REST API with Spring Boot	42
20.4 Spring Data JPA	43

20.5 The Broader Spring Ecosystem	43
Chapter 21: Building Microservices with Java	44
21.1 Monoliths vs Microservices	44
21.2 Service Communication	44
21.3 Key Patterns	44
21.4 Containerization and Orchestration	44
Chapter 22: Interview Preparation and Professional Best Practices	45
22.1 Commonly Asked Conceptual Questions	45
22.2 Practical Coding Interview Tips.....	45
22.3 Writing Production-Quality Code	45
22.4 Continuing Your Java Journey.....	46
Chapter 23: Conclusion	47
Chapter Review: Introduction to Java	47
Chapter Review: Java Fundamentals	47
Chapter Review: Control Flow	48
Chapter Review: Arrays and Strings	48
Chapter Review: OOP — Classes and Objects	48
Chapter Review: OOP — Inheritance and Polymorphism	48
Chapter Review: Exception Handling	49
Chapter Review: Collections Framework.....	49
Chapter Review: Generics.....	49
Chapter Review: Multithreading and Concurrency	50
Chapter Review: File I/O and Serialization	50
Chapter Review: Modern Java — Lambdas, Streams, Optional	50
Chapter Review: Database Connectivity with JDBC	51
Chapter Review: Design Patterns	51
Chapter Review: Memory Management and Garbage Collection	51
Chapter Review: Annotations and Reflection	51
Chapter Review: Networking in Java	52
Chapter Review: Testing with JUnit.....	52
Chapter Review: Build Tools and Best Practices.....	52
Chapter Review: Spring Framework	53
Chapter Review: Microservices with Java	53
Chapter 24: Enums, Nested Classes, and Varargs.....	54
24.1 Enums	54
24.2 Nested and Inner Classes.....	54
24.3 Varargs	55
Chapter 25: Regular Expressions in Java.....	56
25.1 The Pattern and Matcher Classes	56
25.2 Common Regex Building Blocks.....	56
25.3 Practical Uses.....	56
Chapter 26: The Modern Date and Time API	57

26.1 Why java.time?	57
26.2 Core Classes	57
26.3 Working with Time Zones	57
Chapter 27: Data Structures and Algorithms in Java	58
27.1 Big-O Notation	58
27.2 Recursion	58
27.3 Implementing a Linked List	58
27.4 Sorting Algorithms	59
27.5 Trees	59
Chapter 28: JVM Internals and Class Loading	60
28.1 The Class Loading Process	60
28.2 Linking and Initialization	60
28.3 Bytecode and the JIT Compiler	60
28.4 Java Platform Module System (JPMS)	60
Chapter 29: Security in Java Applications	61
29.1 Common Vulnerabilities	61
29.2 Cryptography Basics.....	61
29.3 Secure Coding Checklist	61
Chapter 30: Performance Tuning and Profiling	62
30.1 Measure Before You Optimize.....	62
30.2 Profiling Tools	62
30.3 Common Performance Pitfalls.....	62
30.4 JVM Tuning Flags	62
Appendix A: Java Keywords Reference	63
Appendix B: Quick Reference — Collections Decision Guide	64
B.1 Choosing a List Implementation	64
B.2 Choosing a Set Implementation	64
B.3 Choosing a Map Implementation	64
B.4 Common Time Complexities.....	64
Chapter 31: Case Study — Building a Library Management System	65
31.1 Project Overview	65
31.2 Domain Model	65
31.3 Custom Exceptions.....	66
31.4 The Library Service.....	66
31.5 Putting It Together	67
31.6 Extending the Project	68
Chapter 32: Case Study — A Task Management REST API with Spring Boot	69
32.1 Project Overview	69
32.2 The Entity	69
32.3 The Repository	69
32.4 The Service Layer	69

32.5 The Controller	70
32.6 Global Exception Handling.....	71
32.7 Testing the Service Layer	71
32.8 Deployment Considerations	72
Appendix C: Glossary of Java Terms.....	73
Core Language	73
Runtime and Memory.....	73
Collections and Generics	73
Concurrency.....	73
Modern Java	74
Ecosystem and Tooling	74

Chapter 1: Introduction to Java

1.1 What is Java?

Java is a high-level, class-based, object-oriented programming language originally developed by James Gosling and his team at Sun Microsystems, released publicly in 1995. Sun Microsystems was later acquired by Oracle Corporation in 2010, which now maintains the language. Java was designed around a simple but powerful idea: "Write Once, Run Anywhere" (WORA). This means code compiled on one platform can run on any other platform that supports Java, without needing to be recompiled.

Today, Java powers everything from enterprise banking systems and Android mobile applications to large-scale distributed systems, cloud-native microservices, and embedded devices. It remains one of the most widely used programming languages in the world, consistently ranking in the top tier of language popularity indexes, and is the backbone of countless production systems at companies of every size.

1.2 Key Features of Java

- Platform Independent: Java code compiles into bytecode that runs on the Java Virtual Machine (JVM), which is available for nearly every operating system.
- Object-Oriented: Everything in Java is modeled around objects and classes, encouraging modular, reusable, and maintainable code.
- Simple and Familiar: Java's syntax is derived from C and C++, making it approachable for developers coming from those backgrounds, while removing complexities such as explicit pointer arithmetic.
- Secure: Java runs inside a managed runtime with a security manager, class loader, and bytecode verifier, reducing common vulnerabilities.
- Robust: Strong compile-time type checking, automatic memory management, and exception handling make Java programs less prone to crashes.
- Multithreaded: Java has built-in support for concurrent programming, allowing multiple threads of execution within a single program.
- High Performance: Just-In-Time (JIT) compilation converts bytecode into optimized native machine code at runtime.
- Distributed: Java has extensive networking libraries that make building distributed applications straightforward.
- Dynamic: Java programs carry substantial runtime type information that supports dynamic linking of new classes.

1.3 JDK, JRE, and JVM Explained

These three acronyms confuse many beginners, so it's worth understanding exactly how they relate to one another.

Java Virtual Machine (JVM)

The JVM is an abstract computing machine that provides a runtime environment in which Java bytecode can execute. It is responsible for loading class files, verifying bytecode, executing the code, and managing memory through automatic garbage collection. Each operating system has its own JVM implementation,

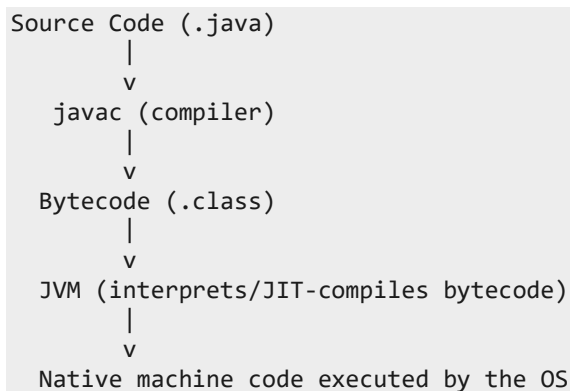
which is what makes Java code portable — the same .class file can run unmodified on Windows, Linux, or macOS as long as a compatible JVM is installed.

Java Runtime Environment (JRE)

The JRE bundles the JVM together with the core class libraries (such as java.lang, java.util, and java.io) and other supporting files needed to run Java applications. If you only need to run Java programs, the JRE alone is sufficient.

Java Development Kit (JDK)

The JDK is a superset of the JRE that additionally includes development tools: the javac compiler, the jar archiving tool, the javadoc documentation generator, debuggers, and profilers. Anyone writing and compiling Java code needs the JDK installed, not just the JRE.



1.4 Setting Up Your Development Environment

To start writing Java, you need a JDK installed and, ideally, an Integrated Development Environment (IDE) such as IntelliJ IDEA, Eclipse, or Visual Studio Code with the Java extension pack. As of recent long-term support (LTS) releases, Java 17 and Java 21 are widely adopted in production environments, and it is recommended that beginners start with the latest LTS version.

Verifying Your Installation

```
$ java -version
java 21.0.2 2024-01-16 LTS

$ javac -version
javac 21.0.2
```

1.5 Your First Java Program

Every Java program begins execution from a special method called main. Let's write the traditional "Hello, World!" program and examine each piece.

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

- `public class HelloWorld` — declares a class named `HelloWorld`; the file must be saved as `HelloWorld.java`, matching the class name exactly.
- `public static void main(String[] args)` — the entry point the JVM looks for when running a program. It must be exactly this signature.
- `System.out.println(...)` — calls the `println` method on the standard output stream to print text followed by a new line.

Compiling and Running

```
$ javac HelloWorld.java
$ java HelloWorld
Hello, World!
```

Tip: *The `javac` command produces a `HelloWorld.class` file containing bytecode. The `java` command then launches the JVM, which loads and executes that bytecode.*

Chapter 2: Java Fundamentals — Syntax, Variables, and Data Types

2.1 Structure of a Java Program

A Java source file typically contains an optional package declaration, optional import statements, and one or more class or interface declarations. Java is case-sensitive, and every statement must end with a semicolon. Curly braces define the scope of classes, methods, and control blocks.

2.2 Variables and Identifiers

A variable is a named piece of memory used to store a value. Java is a statically typed language, meaning every variable must be declared with a specific type before it can be used, and that type cannot change during the variable's lifetime.

```
int age = 25;
double salary = 55000.50;
String name = "Akhil";
boolean isActive = true;
```

Identifiers (the names given to variables, methods, and classes) must begin with a letter, underscore, or dollar sign, and cannot be a reserved keyword. Java convention uses camelCase for variables and methods, and PascalCase for class names.

2.3 Primitive Data Types

Java has eight primitive data types, each with a fixed size and range that does not vary across platforms — a key part of Java's portability guarantee.

- byte — 8-bit signed integer, range -128 to 127
- short — 16-bit signed integer, range -32,768 to 32,767
- int — 32-bit signed integer, the default choice for whole numbers
- long — 64-bit signed integer, used for very large values (suffix with L, e.g. 100000L)
- float — 32-bit floating point (suffix with f, e.g. 3.14f)
- double — 64-bit floating point, the default choice for decimal values
- char — a single 16-bit Unicode character, written in single quotes, e.g. 'A'
- boolean — represents true or false only

2.4 Reference Types and the String Class

Beyond primitives, Java has reference types, which store a reference (memory address) to an object rather than the value itself. Strings, arrays, and every class instance are reference types. The String class deserves special attention because it is immutable — once created, a String object's contents cannot be changed. Operations that appear to modify a string, such as concatenation, actually create a new String object.

```
String greeting = "Hello";
greeting = greeting + ", World!"; // creates a new String object
System.out.println(greeting); // Hello, World!
```

Best Practice: For heavy string manipulation inside loops, prefer `StringBuilder` over repeated `String` concatenation, since `StringBuilder` is mutable and avoids creating many intermediate objects.

2.5 Type Casting and Conversion

Widening conversion happens automatically when converting a smaller type to a larger one (e.g. `int` to `long`) because no data is lost. Narrowing conversion, such as `double` to `int`, requires an explicit cast and may lose precision or data.

```
int i = 100;
long l = i;          // widening, automatic
double d = l;        // widening, automatic

double price = 19.99;
int rounded = (int) price; // narrowing, explicit cast -> 19
```

2.6 Operators

Java supports arithmetic (+ - * / %), relational (== != > < >= <=), logical (&& || !), assignment (= += -= *= /=), and bitwise (& | ^ ~ << >> >>>) operators. The ternary operator (`condition ? valueIfTrue : valueIfFalse`) provides a compact way to write simple conditional expressions.

```
int a = 10, b = 3;
System.out.println(a / b);    // 3 (integer division)
System.out.println(a % b);    // 1 (remainder)
System.out.println(a / (double) b); // 3.3333...

int max = (a > b) ? a : b;    // ternary operator
```

Chapter 3: Control Flow — Decisions and Loops

3.1 Conditional Statements

The if, else if, and else statements allow a program to execute different blocks of code depending on boolean conditions.

```
int score = 82;
if (score >= 90) {
    System.out.println("Grade: A");
} else if (score >= 75) {
    System.out.println("Grade: B");
} else {
    System.out.println("Grade: C or below");
}
```

3.2 The switch Statement and Switch Expressions

The switch statement evaluates a value against a set of possible matches. Modern Java (14+) introduced switch expressions with arrow syntax, which are more concise and avoid the classic fall-through bug caused by forgetting a break statement.

```
// Traditional switch statement
switch (day) {
    case 1:
        System.out.println("Monday");
        break;
    default:
        System.out.println("Another day");
}

// Modern switch expression
String dayName = switch (day) {
    case 1 -> "Monday";
    case 2 -> "Tuesday";
    default -> "Unknown";
};
```

3.3 Loops

Java provides three primary loop constructs: for, while, and do-while, plus the enhanced for-each loop for iterating over arrays and collections.

```
// Standard for loop
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}

// While loop
int count = 0;
while (count < 5) {
    System.out.println(count);
    count++;
}
```

```
// Do-while loop (executes at least once)
int x = 0;
do {
    System.out.println(x);
    x++;
} while (x < 5);

// Enhanced for-each loop
int[] numbers = {1, 2, 3, 4, 5};
for (int n : numbers) {
    System.out.println(n);
}
```

3.4 break, continue, and Labeled Loops

The break statement exits a loop entirely, while continue skips to the next iteration. Labeled loops allow break and continue to target an outer loop from within a nested loop, which is otherwise impossible in Java.

```
outer:
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        if (j == 1) continue outer;
        System.out.println(i + "," + j);
    }
}
```

Chapter 4: Arrays and Strings

4.1 Arrays

An array is a fixed-size, ordered collection of elements of the same type, stored in contiguous memory and accessed by a zero-based index. Once created, an array's length cannot change.

```
int[] numbers = new int[5];           // declaration with default values (all 0)
numbers[0] = 10;
numbers[1] = 20;

int[] scores = {90, 85, 78, 92};      // declaration with initializer
System.out.println(scores.length);    // 4
System.out.println(scores[2]);        // 78
```

4.2 Multidimensional Arrays

Java supports arrays of arrays, commonly used to represent grids or matrices.

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
for (int[] row : matrix) {
    for (int value : row) {
        System.out.print(value + " ");
    }
    System.out.println();
}
```

4.3 The Arrays Utility Class

The `java.util.Arrays` class provides static helper methods for common array operations such as sorting, searching, filling, and comparing.

```
import java.util.Arrays;

int[] nums = {5, 2, 8, 1};
Arrays.sort(nums);
System.out.println(Arrays.toString(nums)); // [1, 2, 5, 8]
int index = Arrays.binarySearch(nums, 5);  // 2
```

4.4 Deep Dive: The String Class

Strings are among the most frequently used types in Java, and understanding their behavior is essential. Because `String` is immutable, the JVM maintains a special memory region called the String Pool to reuse identical string literals and save memory.

```
String a = "hello";
String b = "hello";
System.out.println(a == b);           // true, both refer to the same pooled literal

String c = new String("hello");
```

```
System.out.println(a == c);           // false, c is a new object on the heap
System.out.println(a.equals(c));      // true, contents are equal
```

Important: Always use `.equals()` to compare the contents of two `String` objects. Using `==` compares references, not content, and can lead to subtle bugs.

4.5 Common String Methods

- `length()` — returns the number of characters
- `charAt(int index)` — returns the character at a given position
- `substring(int start, int end)` — extracts a portion of the string
- `indexOf(String s) / lastIndexOf(String s)` — finds the position of a substring
- `toUpperCase() / toLowerCase()` — case conversion
- `trim() / strip()` — removes leading and trailing whitespace
- `split(String regex)` — splits a string into an array based on a delimiter
- `replace(CharSequence target, CharSequence replacement)` — replaces occurrences
- `format(String format, Object... args)` — builds formatted strings, similar to `printf`

4.6 StringBuilder and StringBuffer

`StringBuilder` is a mutable sequence of characters, ideal for building strings incrementally (e.g. inside loops) without the overhead of creating many intermediate `String` objects. `StringBuffer` is functionally identical but synchronized for thread safety, making it slightly slower and generally unnecessary unless multiple threads mutate the same buffer.

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 5; i++) {
    sb.append(i).append(", ");
}
System.out.println(sb.toString()); // 0, 1, 2, 3, 4,
```

Chapter 5: Object-Oriented Programming — Classes and Objects

5.1 Classes and Objects

A class is a blueprint that defines the state (fields) and behavior (methods) that its objects will have. An object is a concrete instance of a class, created using the new keyword. Object-oriented programming organizes software around these interacting objects rather than functions and logic alone.

```
public class Car {  
    // fields (state)  
    String model;  
    int year;  
  
    // method (behavior)  
    void drive() {  
        System.out.println(model + " is driving.");  
    }  
}  
  
// creating and using an object  
Car myCar = new Car();  
myCar.model = "Tesla Model 3";  
myCar.year = 2024;  
myCar.drive(); // Tesla Model 3 is driving.
```

5.2 Constructors

A constructor initializes a newly created object. It shares the class's name and has no return type. If no constructor is defined, Java provides a default no-argument constructor automatically.

```
public class Car {  
    String model;  
    int year;  
  
    // parameterized constructor  
    public Car(String model, int year) {  
        this.model = model;  
        this.year = year;  
    }  
}  
  
Car car = new Car("Honda Civic", 2023);
```

The this keyword refers to the current object instance and is commonly used to disambiguate between a field name and a constructor/method parameter with the same name.

5.3 Access Modifiers and Encapsulation

Encapsulation is the practice of hiding an object's internal state and requiring all interaction to happen through well-defined methods. Java enforces this through access modifiers.

- public — accessible from any class

- protected — accessible within the same package and by subclasses
- default (package-private) — accessible only within the same package
- private — accessible only within the declaring class

```
public class BankAccount {
    private double balance;

    public double getBalance() {
        return balance;
    }

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }
}
```

Best Practice: Keep fields private and expose controlled access through public getter and setter methods. This lets you validate input and change internal implementation without breaking client code.

5.4 Static vs Instance Members

Instance members belong to each individual object, while static members belong to the class itself and are shared across all instances. Static members are accessed via the class name rather than an object reference.

```
public class Counter {
    static int totalCount = 0;    // shared across all instances
    int id;                     // unique to each instance

    public Counter() {
        totalCount++;
        id = totalCount;
    }
}

Counter c1 = new Counter();
Counter c2 = new Counter();
System.out.println(Counter.totalCount); // 2
```

5.5 The Object Class and Method Overriding

Every class in Java implicitly extends `java.lang.Object`, which provides default implementations of `toString()`, `equals()`, and `hashCode()`. It's considered best practice to override these methods to reflect meaningful equality and readable output for your own classes.

```
public class Point {
    int x, y;

    @Override
    public String toString() {
        return "Point(" + x + ", " + y + ")";
    }
}
```

```
}

@Override
public boolean equals(Object obj) {
    if (this == obj) return true;
    if (!(obj instanceof Point)) return false;
    Point p = (Point) obj;
    return x == p.x && y == p.y;
}

@Override
public int hashCode() {
    return Objects.hash(x, y);
}
}
```

Chapter 6: Object-Oriented Programming — Inheritance, Polymorphism, and Abstraction

6.1 Inheritance

Inheritance allows a class (subclass) to acquire the fields and methods of another class (superclass), promoting code reuse. Java supports single inheritance of classes using the `extends` keyword.

```
public class Animal {
    String name;
    void eat() {
        System.out.println(name + " is eating.");
    }
}

public class Dog extends Animal {
    void bark() {
        System.out.println(name + " is barking.");
    }
}

Dog dog = new Dog();
dog.name = "Rex";
dog.eat();    // inherited from Animal
dog.bark();   // defined in Dog
```

6.2 Method Overriding and `super`

A subclass can override a method inherited from its superclass to provide a specialized implementation. The `super` keyword lets the subclass call the superclass's version of a method or constructor.

```
public class Animal {
    void makeSound() {
        System.out.println("Some generic animal sound");
    }
}

public class Cat extends Animal {
    @Override
    void makeSound() {
        super.makeSound(); // call the parent's version too
        System.out.println("Meow");
    }
}
```

6.3 Polymorphism

Polymorphism means "many forms" — the ability for a single interface or method call to behave differently depending on the actual object type at runtime. Java supports two kinds: compile-time polymorphism (method overloading) and runtime polymorphism (method overriding via dynamic dispatch).

```
Animal myPet = new Cat(); // reference type Animal, actual object Cat
myPet.makeSound();        // calls Cat's overridden version at runtime

// Method overloading (compile-time polymorphism)
public int add(int a, int b) { return a + b; }
public double add(double a, double b) { return a + b; }
public int add(int a, int b, int c) { return a + b + c; }
```

6.4 Abstraction: Abstract Classes and Interfaces

Abstraction hides implementation details and exposes only essential behavior. Java achieves this through abstract classes and interfaces.

Abstract Classes

An abstract class cannot be instantiated directly and may contain both abstract methods (no body, must be implemented by subclasses) and concrete methods with full implementations.

```
public abstract class Shape {
    abstract double area(); // must be implemented by subclasses

    void describe() {
        System.out.println("This shape has an area of " + area());
    }
}

public class Circle extends Shape {
    double radius;
    Circle(double radius) { this.radius = radius; }

    @Override
    double area() { return Math.PI * radius * radius; }
}
```

Interfaces

An interface defines a contract of methods that implementing classes must provide. Since Java 8, interfaces can also include default methods (with a body) and static methods. A class can implement multiple interfaces, which is how Java approximates multiple inheritance of behavior.

```
public interface Payable {
    double calculatePayment();

    default void printReceipt() {
        System.out.println("Payment due: " + calculatePayment());
    }
}

public class Invoice implements Payable {
    double amount;
    Invoice(double amount) { this.amount = amount; }

    @Override
    public double calculatePayment() { return amount; }
}
```

Design Tip: *Favor interfaces to define capabilities (what an object can do) and abstract classes to share common state and partial implementation across closely related types.*

6.5 Composition Over Inheritance

While inheritance is powerful, overusing it can lead to fragile, tightly coupled hierarchies. Many experienced Java developers favor composition — building classes by combining smaller, focused objects — over deep inheritance chains, because it results in more flexible and testable designs.

```
public class Engine {
    void start() { System.out.println("Engine starting..."); }
}

public class Car {
    private final Engine engine = new Engine(); // composition

    void start() {
        engine.start();
        System.out.println("Car is ready to drive.");
    }
}
```

Chapter 7: Exception Handling

7.1 What Are Exceptions?

An exception is an event that disrupts the normal flow of a program's instructions, typically caused by an error condition such as invalid input, a missing file, or a network failure. Java uses a robust exception-handling model built around the keywords `try`, `catch`, `finally`, `throw`, and `throws`.

7.2 The Exception Hierarchy

All exceptions descend from `java.lang.Throwable`, which branches into two main subclasses: `Error`, representing serious problems an application typically cannot recover from (such as `OutOfMemoryError`), and `Exception`, representing conditions a well-written program should anticipate and handle.

- Checked Exceptions — subclasses of `Exception` (excluding `RuntimeException`) that must be either caught or declared with `throws`, e.g. `IOException`, `SQLException`.
- Unchecked Exceptions — subclasses of `RuntimeException` that the compiler does not force you to handle, e.g. `NullPointerException`, `ArrayIndexOutOfBoundsException`, `IllegalArgumentException`.

7.3 try-catch-finally

```
public void readFile(String path) {
    try {
        FileReader reader = new FileReader(path);
        // read the file...
    } catch (FileNotFoundException e) {
        System.out.println("File not found: " + e.getMessage());
    } catch (IOException e) {
        System.out.println("I/O error: " + e.getMessage());
    } finally {
        System.out.println("Cleanup code runs whether or not an exception
occurred.");
    }
}
```

7.4 try-with-resources

Introduced in Java 7, `try-with-resources` automatically closes any resource that implements `AutoCloseable` (such as streams, readers, and database connections) once the block finishes, eliminating the need for manual cleanup in a `finally` block.

```
try (BufferedReader reader = new BufferedReader(new FileReader("data.txt"))) {
    String line;
    while ((line = reader.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException e) {
    System.out.println("Error reading file: " + e.getMessage());
}
```

7.5 Custom Exceptions

Applications often define their own exception classes to represent domain-specific error conditions clearly, improving readability and enabling precise handling.

```
public class InsufficientFundsException extends Exception {
    public InsufficientFundsException(String message) {
        super(message);
    }
}

public class BankAccount {
    private double balance;

    public void withdraw(double amount) throws InsufficientFundsException {
        if (amount > balance) {
            throw new InsufficientFundsException("Insufficient balance for
withdrawal");
        }
        balance -= amount;
    }
}
```

7.6 Best Practices

- Catch specific exception types rather than a broad catch (Exception e), so you don't accidentally swallow unrelated bugs.
- Never leave a catch block empty; at minimum, log the exception.
- Only use checked exceptions for recoverable conditions the caller can reasonably act on.
- Include meaningful, actionable messages when throwing exceptions.
- Avoid using exceptions for ordinary control flow — they are relatively expensive and reduce readability when misused.

Chapter 8: The Java Collections Framework

8.1 Overview

The Collections Framework provides a unified architecture for storing and manipulating groups of objects. It is built around a set of core interfaces — Collection, List, Set, Queue, and Map — each with multiple concrete implementations optimized for different use cases.

8.2 List

A List is an ordered collection that permits duplicate elements and allows indexed access.

- ArrayList — backed by a resizable array; fast random access ($O(1)$), slower insertion/removal in the middle ($O(n)$).
- LinkedList — backed by a doubly linked list; fast insertion/removal at the ends ($O(1)$), slower random access ($O(n)$).

```
List<String> names = new ArrayList<>();
names.add("Alice");
names.add("Bob");
names.add(0, "Zoe"); // insert at index 0
System.out.println(names); // [Zoe, Alice, Bob]
```

8.3 Set

A Set is a collection that contains no duplicate elements.

- HashSet — no guaranteed order, $O(1)$ average add/contains, backed by a hash table.
- LinkedHashSet — maintains insertion order while still preventing duplicates.
- TreeSet — maintains elements in sorted order, backed by a red-black tree, $O(\log n)$ operations.

```
Set<String> uniqueNames = new HashSet<>();
uniqueNames.add("Alice");
uniqueNames.add("Alice"); // ignored, duplicate
System.out.println(uniqueNames.size()); // 1
```

8.4 Map

A Map stores key-value pairs, where each key maps to exactly one value. It is not technically part of the Collection interface hierarchy but is a core part of the framework.

- HashMap — no guaranteed order, allows one null key, $O(1)$ average get/put.
- LinkedHashMap — maintains insertion (or access) order.
- TreeMap — maintains keys in sorted order.

```
Map<String, Integer> ages = new HashMap<>();
ages.put("Akhil", 28);
ages.put("Priya", 24);
for (Map.Entry<String, Integer> entry : ages.entrySet()) {
    System.out.println(entry.getKey() + " is " + entry.getValue());
}
```

```
ages.getOrDefault("Unknown", 0); // returns 0 if key absent
```

8.5 Queue and Deque

A Queue models a first-in-first-out (FIFO) collection typically used for holding elements prior to processing. A Deque (double-ended queue) supports insertion and removal at both ends and can also serve as a stack (last-in-first-out).

```
Queue<String> queue = new LinkedList<>();
queue.offer("task1");
queue.offer("task2");
System.out.println(queue.poll()); // task1

Deque<Integer> stack = new ArrayDeque<>();
stack.push(1);
stack.push(2);
System.out.println(stack.pop()); // 2
```

8.6 Choosing the Right Collection

Choosing correctly has real performance consequences. As a rule of thumb: use ArrayList for most list needs unless you're doing frequent insertions/removals at arbitrary positions; use HashMap unless you need sorted or insertion-ordered keys; use HashSet for uniqueness checks; and use ArrayDeque over the legacy Stack and Vector classes, which carry unnecessary synchronization overhead.

8.7 Sorting with Comparable and Comparator

The Comparable interface defines a class's natural ordering via compareTo(), while Comparator defines external, reusable ordering logic, useful when you need multiple ways to sort the same type or can't modify the class itself.

```
public class Employee implements Comparable<Employee> {
    String name;
    double salary;

    @Override
    public int compareTo(Employee other) {
        return Double.compare(this.salary, other.salary);
    }
}

List<Employee> employees = new ArrayList<>();
Collections.sort(employees); // uses compareTo (natural ordering)

// Using a Comparator for custom ordering
employees.sort(Comparator.comparing((Employee e) -> e.name).reversed());
```

Chapter 9: Generics

9.1 Why Generics?

Generics allow classes, interfaces, and methods to operate on types specified by the caller, providing compile-time type safety and eliminating the need for explicit casting. Before generics (pre-Java 5), collections stored plain Object references, which meant type errors surfaced only at runtime.

```
// Without generics (old style, unsafe)
List list = new ArrayList();
list.add("hello");
String s = (String) list.get(0); // manual cast required

// With generics (type-safe)
List<String> list2 = new ArrayList<>();
list2.add("hello");
String s2 = list2.get(0); // no cast needed, compiler enforces type
```

9.2 Generic Classes

```
public class Box<T> {
    private T content;

    public void set(T content) { this.content = content; }
    public T get() { return content; }
}

Box<String> stringBox = new Box<>();
stringBox.set("Hello");

Box<Integer> intBox = new Box<>();
intBox.set(42);
```

9.3 Generic Methods

```
public static <T> void printArray(T[] array) {
    for (T element : array) {
        System.out.println(element);
    }
}

Integer[] intArray = {1, 2, 3};
String[] strArray = {"a", "b", "c"};
printArray(intArray);
printArray(strArray);
```

9.4 Bounded Type Parameters and Wildcards

Bounded type parameters restrict what types can be used, while wildcards (?) provide flexibility when writing methods that operate on collections of an unknown type.

```
// Bounded type parameter: T must be a Number or subtype
public static <T extends Number> double sum(List<T> list) {
    double total = 0;
```

```

    for (T item : list) {
        total += item.doubleValue();
    }
    return total;
}

// Wildcards
public void printAll(List<?> list) {           // unknown type, read-only
    list.forEach(System.out::println);
}
public void addNumbers(List<? super Integer> list) { // accepts Integer or supertype
    list.add(10);
}

```

Mnemonic: PECS: Producer Extends, Consumer Super. Use <? extends T> when you only read from a structure, and <? super T> when you only write to it.

Chapter 10: Multithreading and Concurrency

10.1 Threads and Processes

A process is an independent program with its own memory space, while a thread is a lightweight unit of execution within a process that shares memory with other threads in the same process. Java has built-in support for multithreading through the Thread class and the Runnable interface.

```
class MyTask implements Runnable {
    @Override
    public void run() {
        System.out.println("Running in thread: " + Thread.currentThread().getName());
    }
}

Thread thread = new Thread(new MyTask());
thread.start(); // never call run() directly – that would execute on the current
thread
```

10.2 The Executor Framework

Manually creating and managing threads does not scale well. The java.util.concurrent package introduces the Executor framework, which manages a pool of reusable threads and decouples task submission from execution.

```
ExecutorService executor = Executors.newFixedThreadPool(4);
for (int i = 0; i < 10; i++) {
    int taskId = i;
    executor.submit(() -> System.out.println("Task " + taskId + " on " +
        Thread.currentThread().getName()));
}
executor.shutdown();
```

10.3 Synchronization

When multiple threads access shared mutable state concurrently, race conditions can occur, producing inconsistent results. Java's synchronized keyword ensures that only one thread at a time can execute a critical section of code.

```
public class Counter {
    private int count = 0;

    public synchronized void increment() {
        count++;
    }

    public synchronized int getCount() {
        return count;
    }
}
```

10.4 Locks and Atomic Variables

The `java.util.concurrent.locks` package offers more flexible locking than `synchronized`, such as `ReentrantLock`, which supports try-locking and fairness policies. For simple counters, the `java.util.concurrent.atomic` classes (like `AtomicInteger`) provide lock-free, thread-safe operations.

```
AtomicInteger counter = new AtomicInteger(0);
counter.incrementAndGet();
System.out.println(counter.get());
```

10.5 CompletableFuture

Introduced in Java 8, `CompletableFuture` enables writing asynchronous, non-blocking pipelines of computation that can be composed, combined, and chained.

```
CompletableFuture<String> future = CompletableFuture
    .supplyAsync(() -> "Hello")
    .thenApply(s -> s + ", World!")
    .thenApply(String::toUpperCase);

future.thenAccept(System.out::println); // HELLO, WORLD!
```

10.6 Common Pitfalls

- Deadlock — two or more threads waiting indefinitely for locks held by each other.
- Race conditions — unsynchronized access to shared mutable state producing inconsistent results.
- Thread starvation — a thread perpetually denied access to resources it needs.
- Prefer immutable objects and higher-level concurrency utilities over manual thread management wherever possible.

Chapter 11: File I/O and Serialization

11.1 Streams: Byte vs Character

Java's I/O model is built around streams. Byte streams (InputStream/OutputStream) handle raw binary data, while character streams (Reader/Writer) handle text and correctly manage character encoding.

```
try (FileInputStream fis = new FileInputStream("data.bin")) {
    int byteData;
    while ((byteData = fis.read()) != -1) {
        // process byte
    }
}
```

11.2 Reading and Writing Text Files

```
// Reading with BufferedReader
try (BufferedReader reader = new BufferedReader(new FileReader("input.txt"))) {
    String line;
    while ((line = reader.readLine()) != null) {
        System.out.println(line);
    }
}

// Writing with BufferedWriter
try (BufferedWriter writer = new BufferedWriter(new FileWriter("output.txt"))) {
    writer.write("Hello, file!");
    writer.newLine();
}
```

11.3 The Modern NIO.2 API (java.nio.file)

Since Java 7, the java.nio.file package offers a more powerful and expressive API for file system operations through the Path and Files classes.

```
Path path = Paths.get("output.txt");
Files.writeString(path, "Hello, NIO!");
String content = Files.readString(path);

List<String> lines = Files.readAllLines(path);
boolean exists = Files.exists(path);
```

11.4 Object Serialization

Serialization converts an object's state into a byte stream that can be persisted to a file or sent over a network, and deserialization reverses the process. A class must implement the marker interface Serializable to be eligible.

```
public class User implements Serializable {
    private static final long serialVersionUID = 1L;
    String name;
    int age;
}
```

```
// Serializing
try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("user.ser"))) {
    oos.writeObject(new User());
}

// Deserializing
try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream("user.ser")))
{
    User u = (User) ois.readObject();
}
```

Security Note: *Deserializing data from an untrusted source is a well-known security risk. In modern applications, prefer safer formats such as JSON with a well-vetted library over native Java serialization when handling external input.*

Chapter 12: Modern Java — Lambdas, Streams, and Optional

12.1 Lambda Expressions

Lambda expressions, introduced in Java 8, provide a concise way to represent an instance of a functional interface (an interface with exactly one abstract method) without the boilerplate of an anonymous inner class.

```
// Before Java 8
Runnable r1 = new Runnable() {
    @Override
    public void run() {
        System.out.println("Running");
    }
};

// With a lambda expression
Runnable r2 = () -> System.out.println("Running");

Comparator<String> byLength = (a, b) -> a.length() - b.length();
```

12.2 Functional Interfaces

The `java.util.function` package provides general-purpose functional interfaces used extensively throughout modern Java APIs.

- `Function<T,R>` — takes a T, returns an R
- `Predicate<T>` — takes a T, returns a boolean
- `Consumer<T>` — takes a T, returns nothing
- `Supplier<T>` — takes nothing, returns a T
- `BiFunction<T,U,R>` — takes a T and a U, returns an R

```
Function<Integer, Integer> square = x -> x * x;
Predicate<String> isEmpty = String::isEmpty;
Consumer<String> printer = System.out::println;
Supplier<Double> randomValue = Math::random;
```

12.3 The Stream API

Streams provide a declarative way to process sequences of elements through a pipeline of operations: a source, zero or more intermediate operations (which are lazy and return a new stream), and a terminal operation (which triggers execution and produces a result or side effect).

```
List<String> names = List.of("Alice", "Bob", "Charlie", "Dave", "Eve");

List<String> result = names.stream()
    .filter(name -> name.length() > 3)
    .map(String::toUpperCase)
    .sorted()
    .collect(Collectors.toList());

System.out.println(result); // [ALICE, CHARLIE, DAVE]
```

```
long count = names.stream().filter(n -> n.startsWith("A")).count();
double avgLength = names.stream().mapToInt(String::length).average().orElse(0);
```

12.4 Common Stream Operations

- `filter(Predicate)` — keeps elements matching a condition
- `map(Function)` — transforms each element
- `sorted()` — sorts elements naturally or with a `Comparator`
- `distinct()` — removes duplicates
- `limit(n) / skip(n)` — truncates the stream
- `reduce(...)` — combines elements into a single result
- `collect(Collectors.toList()/toSet()/toMap()/groupingBy())` — accumulates results into a collection

```
Map<Integer, List<String>> byLength = names.stream()
    .collect(Collectors.groupingBy(String::length));

int totalChars = names.stream()
    .mapToInt(String::length)
    .reduce(0, Integer::sum);
```

12.5 Optional

`Optional<T>` is a container object that may or may not hold a non-null value, designed to reduce `NullPointerException` bugs and make the possibility of "no value" explicit in method signatures.

```
Optional<String> maybeName = Optional.ofNullable(getNameOrNull());

String result = maybeName
    .map(String::toUpperCase)
    .orElse("UNKNOWN");

maybeName.ifPresent(name -> System.out.println("Found: " + name));
```

12.6 Records (Java 16+)

Records provide a compact syntax for immutable data-carrier classes, automatically generating a constructor, accessors, `equals()`, `hashCode()`, and `toString()`.

```
public record Point(int x, int y) {}

Point p = new Point(3, 4);
System.out.println(p.x());           // 3
System.out.println(p);                // Point[x=3, y=4]
```

Chapter 13: Database Connectivity with JDBC

13.1 What is JDBC?

Java Database Connectivity (JDBC) is a standard API that allows Java applications to interact with relational databases through a uniform interface, regardless of the specific database vendor, as long as a compatible JDBC driver is available.

13.2 Core JDBC Workflow

```
String url = "jdbc:mysql://localhost:3306/mydb";
String user = "root";
String password = "secret";

try (Connection conn = DriverManager.getConnection(url, user, password);
    PreparedStatement stmt = conn.prepareStatement(
        "SELECT id, name, salary FROM employees WHERE department = ?")) {

    stmt.setString(1, "Engineering");

    try (ResultSet rs = stmt.executeQuery()) {
        while (rs.next()) {
            System.out.println(rs.getInt("id") + ": " + rs.getString("name"));
        }
    }
} catch (SQLException e) {
    e.printStackTrace();
}
```

13.3 PreparedStatement vs Statement

Always prefer PreparedStatement over plain Statement when a query includes user-supplied values. PreparedStatement precompiles the SQL and binds parameters safely, preventing SQL injection attacks — a critical security practice in any production system.

13.4 Transactions

By default, JDBC connections operate in auto-commit mode, where each statement is committed immediately. For multi-step operations that must succeed or fail together, disable auto-commit and manage the transaction explicitly.

```
conn.setAutoCommit(false);
try {
    // multiple related statements
    conn.commit();
} catch (SQLException e) {
    conn.rollback();
} finally {
    conn.setAutoCommit(true);
}
```

13.5 Connection Pooling

Opening a new database connection is expensive. Production applications use a connection pool (such as HikariCP) that maintains a set of reusable connections, dramatically improving throughput under load. Most modern Java applications interact with databases through a higher-level abstraction such as JPA/Hibernate or Spring Data, which internally use JDBC and connection pooling.

Chapter 14: Design Patterns in Java

14.1 Why Design Patterns Matter

Design patterns are proven, reusable solutions to recurring software design problems. They provide a shared vocabulary that helps teams communicate architecture decisions clearly. This chapter covers several patterns that appear constantly in professional Java codebases.

14.2 Singleton Pattern

Ensures a class has only one instance and provides a global point of access to it. Commonly used for configuration managers, logging, and connection pools.

```
public class ConfigManager {
    private static volatile ConfigManager instance;

    private ConfigManager() {}

    public static ConfigManager getInstance() {
        if (instance == null) {
            synchronized (ConfigManager.class) {
                if (instance == null) {
                    instance = new ConfigManager();
                }
            }
        }
        return instance;
    }
}
```

14.3 Factory Pattern

Delegates object creation to a factory method, decoupling client code from concrete classes.

```
public interface Notification {
    void notifyUser();
}

public class NotificationFactory {
    public static Notification create(String type) {
        return switch (type) {
            case "SMS" -> new SmsNotification();
            case "EMAIL" -> new EmailNotification();
            default -> throw new IllegalArgumentException("Unknown type");
        };
    }
}
```

14.4 Builder Pattern

Separates the construction of a complex object from its representation, useful when a class has many optional parameters.

```

public class Pizza {
    private final String size;
    private final boolean cheese;
    private final boolean pepperoni;

    private Pizza(Builder b) {
        this.size = b.size;
        this.cheese = b.cheese;
        this.pepperoni = b.pepperoni;
    }

    public static class Builder {
        private String size = "medium";
        private boolean cheese = false;
        private boolean pepperoni = false;

        public Builder size(String size) { this.size = size; return this; }
        public Builder cheese(boolean cheese) { this.cheese = cheese; return this; }
        public Builder pepperoni(boolean p) { this.pepperoni = p; return this; }
        public Pizza build() { return new Pizza(this); }
    }
}

Pizza pizza = new Pizza.Builder().size("large").cheese(true).build();

```

14.5 Observer Pattern

Defines a one-to-many dependency so that when one object (the subject) changes state, all its dependents (observers) are notified automatically. This underlies event listeners and reactive systems.

```

public interface Observer {
    void update(String event);
}

public class EventPublisher {
    private final List<Observer> observers = new ArrayList<>();

    public void subscribe(Observer o) { observers.add(o); }
    public void publish(String event) {
        observers.forEach(o -> o.update(event));
    }
}

```

14.6 Strategy Pattern

Defines a family of interchangeable algorithms and lets client code select one at runtime, avoiding large conditional blocks.

```

public interface DiscountStrategy {
    double applyDiscount(double price);
}

public class Checkout {
    private final DiscountStrategy strategy;
    public Checkout(DiscountStrategy strategy) { this.strategy = strategy; }
}

```

```
    public double total(double price) { return strategy.applyDiscount(price); }  
}
```

14.7 Dependency Injection

Rather than a class constructing its own dependencies, they are supplied ('injected') from outside — typically via constructor parameters. This decouples classes from concrete implementations and is the foundation of frameworks like Spring.

```
public class OrderService {  
    private final PaymentGateway gateway;  
  
    // dependency injected via constructor  
    public OrderService(PaymentGateway gateway) {  
        this.gateway = gateway;  
    }  
}
```

Chapter 15: Java Memory Management and Garbage Collection

15.1 The JVM Memory Model

The JVM divides memory into several regions during execution.

- Heap — where all objects and arrays are allocated; further divided into Young Generation (Eden + Survivor spaces) and Old Generation.
- Stack — one per thread, stores local variables and method call frames; automatically reclaimed when a method returns.
- Method Area / Metaspace — stores class metadata, static variables, and constant pool data.
- PC Registers and Native Method Stacks — support low-level execution bookkeeping.

15.2 Garbage Collection Fundamentals

Java automatically reclaims memory occupied by objects that are no longer reachable from any live reference, freeing developers from manual memory management (unlike C/C++). Most objects are short-lived and allocated in the Young Generation; objects that survive multiple garbage collection cycles are eventually promoted to the Old Generation.

15.3 Garbage Collector Algorithms

- Serial GC — single-threaded, suitable for small applications with limited memory.
- Parallel GC — uses multiple threads for collection, optimized for throughput.
- G1 (Garbage-First) GC — the default collector since Java 9, balances throughput and pause times by dividing the heap into regions.
- ZGC and Shenandoah — low-latency collectors designed for very large heaps with sub-millisecond pause times, increasingly used in latency-sensitive services.

15.4 Memory Leaks in Java

Even with automatic garbage collection, memory leaks can still occur when references to unused objects are unintentionally retained — for example, in static collections, unclosed resources, or improperly implemented caches. Tools like VisualVM, JProfiler, and Java Flight Recorder help diagnose these issues in production.

15.5 Tuning Tips

- Set `-Xms` and `-Xmx` to configure initial and maximum heap size appropriately for your workload.
- Choose a garbage collector suited to your latency and throughput requirements with flags like `-XX:+UseG1GC`.
- Avoid creating unnecessary objects in hot code paths, especially inside loops.
- Close resources promptly using `try-with-resources` to avoid leaking file handles, sockets, or connections.

Chapter 16: Annotations and Reflection

16.1 Built-in Annotations

Annotations attach metadata to code without directly affecting its execution. Java includes several built-in annotations.

- `@Override` — indicates a method overrides a superclass or interface method; the compiler verifies this.
- `@Deprecated` — marks a program element as discouraged for use.
- `@SuppressWarnings` — instructs the compiler to suppress specific warnings.
- `@FunctionalInterface` — documents and enforces that an interface has exactly one abstract method.

16.2 Custom Annotations

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Loggable {
    String value() default "";
}

public class Service {
    @Loggable("important operation")
    public void process() { /* ... */ }
}
```

16.3 Reflection

Reflection is the ability of a program to inspect and manipulate its own classes, methods, and fields at runtime. It underpins many frameworks (Spring, Hibernate, JUnit) that need to instantiate objects and invoke methods dynamically without compile-time knowledge of the specific classes involved.

```
Class<?> clazz = Class.forName("com.example.Service");
Object instance = clazz.getDeclaredConstructor().newInstance();

Method method = clazz.getMethod("process");
method.invoke(instance);

for (Method m : clazz.getDeclaredMethods()) {
    if (m.isAnnotationPresent(Loggable.class)) {
        System.out.println("Found loggable method: " + m.getName());
    }
}
```

Caution: *Reflection is powerful but comes with performance overhead and bypasses normal compile-time type checking. It should be used deliberately — typically inside frameworks and libraries — rather than in everyday application code.*

Chapter 17: Networking in Java

17.1 Sockets

The `java.net` package provides classes for low-level network programming. A `Socket` represents a client-side connection endpoint, while a `ServerSocket` listens for and accepts incoming connections on the server side.

```
// Simple server
try (ServerSocket serverSocket = new ServerSocket(8080)) {
    Socket clientSocket = serverSocket.accept();
    BufferedReader in = new BufferedReader(new
InputStreamReader(clientSocket.getInputStream()));
    PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
    out.println("Hello from server: " + in.readLine());
}

// Simple client
try (Socket socket = new Socket("localhost", 8080)) {
    PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
    out.println("Hello, server!");
}
```

17.2 HTTP Clients

Java 11 introduced a modern `java.net.http.HttpClient` for making HTTP requests without third-party libraries, supporting synchronous and asynchronous calls, HTTP/2, and WebSockets.

```
HttpClient client = HttpClient.newHttpClient();
HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://api.example.com/data"))
    .GET()
    .build();

HttpResponse<String> response = client.send(request,
HttpResponse.BodyHandlers.ofString());
System.out.println(response.statusCode());
System.out.println(response.body());
```

17.3 RESTful Web Services

In modern professional Java development, most networked applications are built as RESTful web services using frameworks like Spring Boot or Jakarta EE (JAX-RS), which handle the low-level socket and HTTP details and let developers focus on defining resources, request mappings, and business logic.

Chapter 18: Testing Java Applications with JUnit

18.1 Why Automated Testing Matters

Automated tests catch regressions early, document expected behavior, and give developers the confidence to refactor code safely. JUnit is the de facto standard testing framework in the Java ecosystem, with JUnit 5 (Jupiter) being the current major version used in most modern projects.

18.2 Writing a Basic Test

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CalculatorTest {
    @Test
    void addsTwoNumbers() {
        Calculator calc = new Calculator();
        assertEquals(5, calc.add(2, 3));
    }

    @Test
    void throwsOnDivideByZero() {
        Calculator calc = new Calculator();
        assertThrows(ArithmeticException.class, () -> calc.divide(10, 0));
    }
}
```

18.3 Lifecycle Annotations

- `@BeforeEach` / `@AfterEach` — run before/after every test method
- `@BeforeAll` / `@AfterAll` — run once before/after all tests in the class (must be static)
- `@Disabled` — temporarily skips a test
- `@ParameterizedTest` — runs the same test logic with multiple different inputs

```
@ParameterizedTest
@ValueSource(ints = {1, 2, 3, 4, 5})
void isPositive(int number) {
    assertTrue(number > 0);
}
```

18.4 Mocking with Mockito

Real-world classes often depend on external services, databases, or other collaborators that are impractical to use directly in a unit test. Mockito lets you create mock objects that simulate this behavior in isolation.

```
@Test
void processesOrderSuccessfully() {
    PaymentGateway mockGateway = mock(PaymentGateway.class);
    when(mockGateway.charge(anyDouble())).thenReturn(true);

    OrderService service = new OrderService(mockGateway);
}
```

```
    assertTrue(service.placeOrder(99.99));  
  
    verify(mockGateway).charge(99.99);  
}
```

18.5 The Testing Pyramid

Professional teams typically aim for a large base of fast unit tests, a smaller layer of integration tests verifying components work together (including real databases or message queues via tools like Testcontainers), and a thin top layer of end-to-end tests validating complete user flows.

Chapter 19: Build Tools and Professional Best Practices

19.1 Maven

Maven is a build automation and dependency management tool driven by a declarative `pom.xml` file. It standardizes project structure, resolves dependencies from remote repositories, and defines a build lifecycle (validate, compile, test, package, verify, install, deploy).

```
<project>
  <groupId>com.example</groupId>
  <artifactId>my-app</artifactId>
  <version>1.0.0</version>
  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.10.0</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

19.2 Gradle

Gradle is a more flexible, script-based build tool (using Groovy or Kotlin DSL) that has become the default for many modern projects, including all Android development, largely due to faster incremental builds and more expressive configuration.

```
plugins {
    id 'java'
}

dependencies {
    testImplementation 'org.junit.jupiter:junit-jupiter:5.10.0'
}
```

19.3 Code Style and Readability

- Follow consistent naming conventions: PascalCase for classes, camelCase for methods and variables, UPPER_SNAKE_CASE for constants.
- Keep methods short and focused on a single responsibility.
- Write self-documenting code; use comments to explain why, not what.
- Use tools like Checkstyle, PMD, or SpotBugs to enforce style and catch bugs automatically.

19.4 Version Control Discipline

- Write clear, focused commit messages that explain the intent of a change.
- Keep commits small and logically scoped rather than bundling unrelated changes together.
- Use feature branches and pull requests to enable code review before merging.
- Never commit secrets, credentials, or environment-specific configuration to source control.

19.5 Logging

Production applications should never rely on `System.out.println` for diagnostics. Use a structured logging framework such as SLF4J with Logback or Log4j2, which supports log levels, structured output, and configurable destinations.

```
private static final Logger logger = LoggerFactory.getLogger(OrderService.class);

logger.info("Processing order {}", orderId);
logger.error("Payment failed for order {}", orderId, exception);
```

Chapter 20: Introduction to the Spring Framework

20.1 Why Spring?

Spring is the most widely used application framework in the Java enterprise ecosystem. At its core, Spring provides Inversion of Control (IoC) and Dependency Injection, which decouple components and make applications easier to test and maintain. Spring Boot, built on top of Spring, drastically simplifies configuration through auto-configuration and embedded servers, letting developers go from zero to a running application in minutes.

20.2 The IoC Container and Beans

Objects managed by Spring are called beans. Instead of your code creating dependencies with `new`, you declare what a class needs, and the Spring container wires everything together at startup.

```
@Component
public class EmailService {
    public void send(String to, String message) {
        System.out.println("Sending email to " + to);
    }
}

@Service
public class OrderService {
    private final EmailService emailService;

    @Autowired
    public OrderService(EmailService emailService) {
        this.emailService = emailService;
    }
}
```

20.3 Building a REST API with Spring Boot

```
@RestController
@RequestMapping("/api/orders")
public class OrderController {

    private final OrderService orderService;

    public OrderController(OrderService orderService) {
        this.orderService = orderService;
    }

    @GetMapping("/{id}")
    public ResponseEntity<Order> getOrder(@PathVariable Long id) {
        return ResponseEntity.ok(orderService.findById(id));
    }

    @PostMapping
    public ResponseEntity<Order> createOrder(@RequestBody Order order) {
        Order saved = orderService.save(order);
        return ResponseEntity.status(HttpStatus.CREATED).body(saved);
    }
}
```

```
}  
}
```

20.4 Spring Data JPA

Spring Data JPA eliminates most boilerplate data-access code by generating implementations of repository interfaces at runtime, based on method naming conventions or annotated queries.

```
public interface OrderRepository extends JpaRepository<Order, Long> {  
    List<Order> findByCustomerName(String customerName);  
    List<Order> findByStatusAndTotalGreaterThan(String status, double total);  
}
```

20.5 The Broader Spring Ecosystem

- Spring Security — authentication and authorization
- Spring Cloud — tools for building distributed systems and microservices
- Spring Batch — large-scale batch processing
- Spring WebFlux — reactive, non-blocking web applications

Chapter 21: Building Microservices with Java

21.1 Monoliths vs Microservices

A monolithic application is deployed as a single unit, while a microservices architecture decomposes an application into small, independently deployable services, each owning its own data and communicating over the network — typically via REST or messaging. Microservices offer independent scalability and deployment at the cost of increased operational complexity.

21.2 Service Communication

Services typically communicate synchronously via REST or gRPC, or asynchronously via message brokers such as Kafka or RabbitMQ, which decouple producers and consumers and improve resilience to partial failures.

21.3 Key Patterns

- API Gateway — a single entry point that routes requests to the appropriate backend service.
- Service Discovery — allows services to find each other dynamically as instances scale up or down (e.g. Eureka, Consul).
- Circuit Breaker — prevents cascading failures by short-circuiting calls to a failing downstream service (e.g. Resilience4j).
- Centralized Configuration — externalizes configuration so it can change without redeploying services (e.g. Spring Cloud Config).
- Distributed Tracing — tracks a single request as it flows across multiple services (e.g. OpenTelemetry, Zipkin).

21.4 Containerization and Orchestration

Java microservices are commonly packaged as Docker containers and orchestrated with Kubernetes, which handles scaling, self-healing, and rolling deployments. Spring Boot applications can be containerized easily using layered JARs or tools like Jib, which build optimized container images without requiring a separate Dockerfile.

Chapter 22: Interview Preparation and Professional Best Practices

22.1 Commonly Asked Conceptual Questions

- What is the difference between `==` and `.equals()`? — `==` compares references (or primitive values); `.equals()` compares logical content, and can be overridden.
- What is the difference between an abstract class and an interface? — An abstract class can hold state and constructors and supports single inheritance; an interface defines a contract, supports multiple implementation, and (since Java 8) can include default methods.
- What is the difference between `HashMap` and `Hashtable`? — `HashMap` is unsynchronized and allows one null key; `Hashtable` is legacy, synchronized, and does not allow null keys or values.
- Explain the difference between checked and unchecked exceptions and when to use each.
- What happens during garbage collection, and what are the generational hypothesis assumptions it relies on?
- Explain how `HashMap` works internally, including hashing, buckets, and collision resolution via linked lists or balanced trees.
- What is the difference between `final`, `finally`, and `finalize()`? — `final` is a modifier preventing reassignment/overriding/extension; `finally` is a block that always executes after `try/catch`; `finalize()` was a deprecated method called before garbage collection.

22.2 Practical Coding Interview Tips

- Practice implementing common data structures (linked lists, trees, hash maps) from scratch to solidify your understanding of what the standard library abstracts away.
- Be ready to reason about time and space complexity (Big-O) for any solution you propose.
- Communicate your thought process out loud; interviewers usually care as much about approach as the final answer.
- Review the Collections Framework decision tree — knowing when to use a `List` vs `Set` vs `Map` vs `Deque` is one of the most common practical assessments.

22.3 Writing Production-Quality Code

- Favor immutability where practical; immutable objects are inherently thread-safe and easier to reason about.
- Validate inputs at boundaries (e.g. public API methods) and fail fast with clear exceptions.
- Write unit tests alongside your code, not as an afterthought.
- Document public APIs with Javadoc comments so IDEs can surface useful information to consumers of your code.
- Profile before optimizing — don't guess at performance bottlenecks; measure them.

22.4 Continuing Your Java Journey

Mastery comes from building real projects, reading other people's code, and gradually taking on more complex systems. Contributing to open-source Java projects, working through the official Java Language Specification for tricky edge cases, and staying current with new language features released roughly every six months are all excellent ways to keep growing as a professional Java developer.

Chapter 23: Conclusion

This book has taken you from the very first "Hello, World!" program through the core language fundamentals, object-oriented design, the collections framework, generics, concurrency, modern functional-style programming with streams and lambdas, and into professional, real-world territory: databases, design patterns, testing, memory management, and enterprise frameworks like Spring and microservices architecture.

Java's greatest strength lies not just in the language itself, but in the ecosystem, tooling, and communities built around it over three decades. The concepts covered here form a solid foundation, but true expertise comes from continued practice — writing code, reading code, debugging real problems, and gradually internalizing the idioms that experienced Java engineers use every day.

Whether you are just starting your first program or preparing to lead architecture decisions on large-scale distributed systems, the fundamentals in this book will remain relevant. Keep building, keep questioning how things work under the hood, and keep learning.

— End of Book —

Chapter Review: Introduction to Java

Test your understanding with the following review questions and short exercises before moving on to the next chapter.

- 1. Explain the difference between the JDK, JRE, and JVM in your own words.
- 2. Why is Java described as 'platform independent'? What role does bytecode play?
- 3. What is the purpose of the main method signature, and why must it match exactly?
- 4. Write and compile a program that prints your name and today's date to the console.
- 5. Research one production system or company you use daily that is known to run on Java.

Chapter Review: Java Fundamentals

Test your understanding with the following review questions and short exercises before moving on to the next chapter.

- 1. List all eight primitive types along with their size in bits and a typical use case for each.
- 2. Explain why String is immutable and what practical benefit this provides.
- 3. What is the difference between widening and narrowing type conversion? Give an example of each.
- 4. Write a short program that swaps the values of two int variables without using a third variable.
- 5. Predict the output of: `System.out.println(7 / 2 + " " + 7 % 2 + " " + (7 / 2.0));`

Chapter Review: Control Flow

Test your understanding with the following review questions and short exercises before moving on to the next chapter.

- 1. Rewrite a chain of if-else-if statements as a modern switch expression.
- 2. Write a program that prints the first 20 Fibonacci numbers using a while loop.
- 3. Explain the difference between break and continue with a concrete example of each.
- 4. When would you reach for a labeled loop instead of a boolean flag variable?
- 5. Write a do-while loop that repeatedly prompts a user until they enter a positive number.

Chapter Review: Arrays and Strings

Test your understanding with the following review questions and short exercises before moving on to the next chapter.

- 1. Write a method that reverses an int array in place without creating a new array.
- 2. Explain why using == to compare two String objects created with new String(...) can produce unexpected results.
- 3. Write a program that counts how many times each character appears in a given string.
- 4. What is the time complexity of accessing an element in an ArrayList by index versus a LinkedList?
- 5. When would you choose StringBuilder over regular String concatenation? Justify with a scenario.

Chapter Review: OOP — Classes and Objects

Test your understanding with the following review questions and short exercises before moving on to the next chapter.

- 1. Design a class Rectangle with private fields width and height, a constructor, getters, and a method to compute area.
- 2. Explain the difference between an instance method and a static method, with an example of when each is appropriate.
- 3. Why should fields typically be private rather than public? What problem does this prevent?
- 4. Override toString(), equals(), and hashCode() for a simple Person class with name and age fields.
- 5. What does the this keyword refer to inside a constructor, and when is it strictly necessary?

Chapter Review: OOP — Inheritance and Polymorphism

Test your understanding with the following review questions and short exercises before moving on to the next chapter.

- 1. Create a class hierarchy: Vehicle -> Car, Motorcycle, each overriding a describe() method.
- 2. Explain the difference between method overloading and method overriding.
- 3. What is dynamic method dispatch, and how does the JVM decide which overridden method to call at runtime?

- 4. When would you choose an interface over an abstract class for a given design problem?
- 5. Refactor a class that inherits from three unrelated concerns into a design that favors composition instead.

Chapter Review: Exception Handling

Test your understanding with the following review questions and short exercises before moving on to the next chapter.

- 1. Write a method that reads a file and gracefully handles both `FileNotFoundException` and `IOException`.
- 2. Create a custom checked exception called `InvalidAgeException` and use it in a method that validates age input.
- 3. Explain the difference between checked and unchecked exceptions and give two examples of each from the standard library.
- 4. Why is an empty catch block considered a bad practice?
- 5. Rewrite a resource-handling try/finally block using try-with-resources.

Chapter Review: Collections Framework

Test your understanding with the following review questions and short exercises before moving on to the next chapter.

- 1. Given a list of employee objects, write code to group them by department using a `Map`.
- 2. Explain when you would choose a `TreeSet` over a `HashSet`.
- 3. Write a `Comparator` that sorts a list of `Strings` by length, and then alphabetically for ties.
- 4. What is the time complexity of `get()` and `put()` in a `HashMap` on average, and why can it degrade?
- 5. Convert a `List<Integer>` into a `Set<Integer>` and explain what happens to duplicate values.

Chapter Review: Generics

Test your understanding with the following review questions and short exercises before moving on to the next chapter.

- 1. Write a generic class `Pair<A, B>` that holds two values of potentially different types.
- 2. Explain PECS (Producer Extends, Consumer Super) with a concrete method signature example.
- 3. Why can't you create a generic array directly in Java (e.g. `new T[10]`)?
- 4. Write a generic method that finds the maximum element in a list of any `Comparable` type.
- 5. What compiler errors would you expect if you tried to add a `String` to a `List<Integer>`?

Chapter Review: Multithreading and Concurrency

Test your understanding with the following review questions and short exercises before moving on to the next chapter.

- 1. Write a program that starts three threads that each print numbers 1 to 5 with their thread name.
- 2. Explain a race condition and demonstrate how synchronized prevents it in a shared counter example.
- 3. What is a deadlock, and what conditions must be present for one to occur?
- 4. Rewrite a manual Thread-based task submission using an ExecutorService with a fixed thread pool.
- 5. When would you prefer AtomicInteger over a synchronized method for a simple counter?

Chapter Review: File I/O and Serialization

Test your understanding with the following review questions and short exercises before moving on to the next chapter.

- 1. Write a program using java.nio.file.Files to read all lines from a text file and print only lines containing a keyword.
- 2. Explain the difference between byte streams and character streams, and when each is appropriate.
- 3. Implement Serializable on a custom class and write code to serialize and deserialize an instance.
- 4. Why is deserializing untrusted data considered a security risk?
- 5. Write code that copies the contents of one file to another using try-with-resources.

Chapter Review: Modern Java — Lambdas, Streams, Optional

Test your understanding with the following review questions and short exercises before moving on to the next chapter.

- 1. Given a List<Employee>, use streams to find the average salary of employees in the 'Engineering' department.
- 2. Rewrite an anonymous Comparator as a lambda expression, then as a method reference if possible.
- 3. Explain the difference between map() and flatMap() in the Stream API.
- 4. Write code using Optional to safely handle a method that might return null, avoiding a NullPointerException.
- 5. Convert a traditional class with fields, a constructor, and getters into an equivalent Java record.

Chapter Review: Database Connectivity with JDBC

Test your understanding with the following review questions and short exercises before moving on to the next chapter.

- 1. Explain why PreparedStatement is preferred over Statement when handling user input.
- 2. Write JDBC code to insert a new row into a table within an explicit transaction, rolling back on failure.

- 3. What is connection pooling, and why does it matter for application performance?
- 4. Describe the difference between `executeQuery()`, `executeUpdate()`, and `execute()`.
- 5. What security risk does SQL injection pose, and how does JDBC help prevent it?

Chapter Review: Design Patterns

Test your understanding with the following review questions and short exercises before moving on to the next chapter.

- 1. Implement a thread-safe Singleton using an enum instead of double-checked locking.
- 2. Design a Factory that creates different Notification implementations based on a configuration string.
- 3. Refactor a class with a telescoping constructor (many overloaded constructors) to use the Builder pattern.
- 4. Explain how the Observer pattern relates to Java's event listener model in GUI frameworks.
- 5. When would the Strategy pattern be a better choice than a series of if-else branches?

Chapter Review: Memory Management and Garbage Collection

Test your understanding with the following review questions and short exercises before moving on to the next chapter.

- 1. Explain the difference between the Young Generation and Old Generation in the JVM heap.
- 2. What are three common causes of memory leaks in Java despite automatic garbage collection?
- 3. Describe at a high level how the G1 garbage collector differs from the Serial collector.
- 4. Why does closing resources via try-with-resources matter even though Java has garbage collection?
- 5. What JVM flags would you use to set a 512MB initial heap and 2GB maximum heap?

Chapter Review: Annotations and Reflection

Test your understanding with the following review questions and short exercises before moving on to the next chapter.

- 1. Create a custom annotation `@Auditable` that can be applied to methods, retained at runtime.
- 2. Write reflection code that lists all public methods of a given class along with their parameter types.
- 3. Explain why reflection has performance overhead compared to direct method calls.
- 4. Name two popular frameworks that rely heavily on reflection and briefly explain why.
- 5. What is the purpose of the `@FunctionalInterface` annotation, and is it strictly required?

Chapter Review: Networking in Java

Test your understanding with the following review questions and short exercises before moving on to the next chapter.

- 1. Write a minimal client-server pair using Socket and ServerSocket that exchanges a single message.
- 2. Use HttpClient to make a GET request to a public API and print the response body.
- 3. Explain the difference between synchronous and asynchronous HTTP calls in java.net.http.HttpClient.
- 4. What advantages does a RESTful web service built with a framework offer over raw sockets?
- 5. Research and summarize the difference between HTTP/1.1 and HTTP/2 in two or three sentences.

Chapter Review: Testing with JUnit

Test your understanding with the following review questions and short exercises before moving on to the next chapter.

- 1. Write a JUnit 5 test class for a Calculator class covering addition, subtraction, and division by zero.
- 2. Convert a set of near-identical test methods into a single @ParameterizedTest.
- 3. Use Mockito to mock a repository dependency and verify a service method calls save() exactly once.
- 4. Explain the testing pyramid and why unit tests should outnumber end-to-end tests.
- 5. What is the difference between @BeforeEach and @BeforeAll, and when would you use each?

Chapter Review: Build Tools and Best Practices

Test your understanding with the following review questions and short exercises before moving on to the next chapter.

- 1. Write a minimal pom.xml for a project that depends on JUnit 5 for testing.
- 2. Explain the Maven build lifecycle phases from compile through deploy.
- 3. Why should logging frameworks be preferred over System.out.println in production code?
- 4. List three habits that make Git commit history easier for a team to review.
- 5. What is the purpose of a linter like Checkstyle or PMD in a CI pipeline?

Chapter Review: Spring Framework

Test your understanding with the following review questions and short exercises before moving on to the next chapter.

- 1. Explain Dependency Injection in your own words and why it improves testability.
- 2. Write a simple @RestController with a GET endpoint that returns a list of products as JSON.
- 3. What problem does Spring Data JPA solve compared to writing raw JDBC code?
- 4. Describe the role of the Spring IoC container in managing bean lifecycles.
- 5. Name three modules in the broader Spring ecosystem and what each is responsible for.

Chapter Review: Microservices with Java

Test your understanding with the following review questions and short exercises before moving on to the next chapter.

- 1. Explain the tradeoffs between a monolithic architecture and a microservices architecture.
- 2. What problem does a Circuit Breaker solve in a distributed system?
- 3. Describe the role of an API Gateway in a microservices architecture.
- 4. Why is distributed tracing important once an application is split into many services?
- 5. Compare synchronous REST communication with asynchronous messaging for inter-service communication.

Chapter 24: Enums, Nested Classes, and Varargs

24.1 Enums

An enum is a special type representing a fixed set of named constants. Java enums are full-fledged classes, meaning they can have fields, constructors, and methods, making them far more powerful than simple integer constants used in many other languages.

```
public enum Day {
    MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY;

    public boolean isWeekend() {
        return this == SATURDAY || this == SUNDAY;
    }
}

Day today = Day.SATURDAY;
System.out.println(today.isWeekend()); // true
```

Enums with Constructors and Fields

```
public enum Planet {
    MERCURY(3.303e+23, 2.4397e6),
    VENUS(4.869e+24, 6.0518e6),
    EARTH(5.976e+24, 6.37814e6);

    private final double mass;
    private final double radius;

    Planet(double mass, double radius) {
        this.mass = mass;
        this.radius = radius;
    }

    double surfaceGravity() {
        final double G = 6.67300E-11;
        return G * mass / (radius * radius);
    }
}
```

24.2 Nested and Inner Classes

Java allows classes to be defined inside other classes. There are four variants, each with different semantics.

- Static nested class — behaves like a top-level class but is namespaced inside the enclosing class; does not hold a reference to an outer instance.
- Inner (non-static) class — holds an implicit reference to its enclosing instance and can access its private members directly.
- Local class — defined within a method body, scoped to that method.
- Anonymous class — a one-off class defined and instantiated in a single expression, often used before lambdas existed.

```

public class Outer {
    private int value = 10;

    class Inner {
        void show() {
            System.out.println("Outer value: " + value); // direct access
        }
    }

    static class StaticNested {
        void show() {
            System.out.println("I don't need an Outer instance.");
        }
    }
}

Outer outer = new Outer();
Outer.Inner inner = outer.new Inner();
inner.show();

```

24.3 Varargs

Variable-length argument lists (varargs) allow a method to accept zero or more arguments of a specified type, treated internally as an array.

```

public static int sum(int... numbers) {
    int total = 0;
    for (int n : numbers) {
        total += n;
    }
    return total;
}

sum();           // 0
sum(1, 2, 3);    // 6
sum(5, 10, 15, 20); // 50

```

Rule: A method can have only one varargs parameter, and it must be the last parameter in the method signature.

Chapter 25: Regular Expressions in Java

25.1 The Pattern and Matcher Classes

Java's `java.util.regex` package provides `Pattern` (a compiled representation of a regular expression) and `Matcher` (an engine that performs match operations against a given input) for powerful text processing.

```
Pattern pattern = Pattern.compile("\\d{3}-\\d{3}-\\d{4}");
Matcher matcher = pattern.matcher("Call me at 555-123-4567 today");
if (matcher.find()) {
    System.out.println("Found: " + matcher.group()); // 555-123-4567
}
```

25.2 Common Regex Building Blocks

- `\\d` — a digit; `\\D` — a non-digit
- `\\w` — a word character; `\\s` — whitespace
- `*` — zero or more; `+` — one or more; `?` — zero or one
- `{n,m}` — between `n` and `m` repetitions
- `^` and `$` — start and end of a line
- `()` — capturing group; `(?:)` — non-capturing group

25.3 Practical Uses

```
// Validating an email format
boolean isValid = "user@example.com".matches("^([\\w.-]+@[\\w.-]+\\.\\.[a-zA-Z]{2,}$");

// Splitting on multiple delimiters
String[] parts = "apple, banana;cherry".split("[,;]\\s*");

// Replacing with a regex
String cleaned = "Hello   World".replaceAll("\\s+", " ");
```

Tip: For anything beyond simple validation, prefer well-tested library methods or dedicated parsers over increasingly complex regular expressions, which quickly become hard to read and maintain.

Chapter 26: The Modern Date and Time API

26.1 Why java.time?

Java 8 introduced the java.time package (JSR-310), replacing the notoriously error-prone and mutable java.util.Date and Calendar classes with an immutable, thread-safe, and far more intuitive API.

26.2 Core Classes

- LocalDate — a date without time or time zone, e.g. 2026-07-10
- LocalTime — a time without date or time zone, e.g. 14:30:00
- LocalDateTime — a combination of date and time, without a time zone
- ZonedDateTime — a date-time with a specific time zone
- Duration — a time-based amount, e.g. '34.5 seconds'
- Period — a date-based amount, e.g. '2 years, 3 months, 4 days'

```
LocalDate today = LocalDate.now();
LocalDate birthday = LocalDate.of(1998, Month.MARCH, 15);

Period age = Period.between(birthday, today);
System.out.println(age.getYears() + " years old");

LocalDateTime meeting = LocalDateTime.of(2026, 7, 15, 14, 30);
LocalDateTime later = meeting.plusHours(2).plusDays(1);

DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MMM-yyyy HH:mm");
System.out.println(later.format(formatter));
```

26.3 Working with Time Zones

```
ZonedDateTime nowInTokyo = ZonedDateTime.now(ZoneId.of("Asia/Tokyo"));
ZonedDateTime nowInNewYork =
    nowInTokyo.withZoneSameInstant(ZoneId.of("America/New_York"));

System.out.println(nowInTokyo);
System.out.println(nowInNewYork);
```

Chapter 27: Data Structures and Algorithms in Java

27.1 Big-O Notation

Big-O notation describes how an algorithm's running time or memory usage grows relative to input size, providing a common vocabulary for comparing algorithmic efficiency independent of hardware.

- $O(1)$ — constant time, e.g. array index access
- $O(\log n)$ — logarithmic, e.g. binary search
- $O(n)$ — linear, e.g. a single loop through a list
- $O(n \log n)$ — typical of efficient sorting algorithms like mergesort
- $O(n^2)$ — quadratic, e.g. nested loops comparing all pairs

27.2 Recursion

A recursive method calls itself to solve smaller instances of the same problem, always requiring a base case to prevent infinite recursion.

```
public static int factorial(int n) {
    if (n <= 1) return 1;        // base case
    return n * factorial(n - 1); // recursive case
}

public static int fibonacci(int n) {
    if (n <= 1) return n;
    return fibonacci(n - 1) + fibonacci(n - 2);
}
```

27.3 Implementing a Linked List

```
public class LinkedList<T> {
    private Node<T> head;

    private static class Node<T> {
        T data;
        Node<T> next;
        Node(T data) { this.data = data; }
    }

    public void add(T data) {
        Node<T> newNode = new Node<>(data);
        if (head == null) { head = newNode; return; }
        Node<T> current = head;
        while (current.next != null) current = current.next;
        current.next = newNode;
    }

    public void print() {
        Node<T> current = head;
        while (current != null) {
            System.out.print(current.data + " -> ");
            current = current.next;
        }
    }
}
```

```

    }
    System.out.println("null");
}
}

```

27.4 Sorting Algorithms

While production code should rely on the highly optimized `Arrays.sort()` and `Collections.sort()` (which use a tuned dual-pivot quicksort for primitives and a stable timsort for objects), understanding classic sorting algorithms builds essential problem-solving intuition.

```

// Bubble sort (educational, O(n^2))
public static void bubbleSort(int[] arr) {
    for (int i = 0; i < arr.length - 1; i++) {
        for (int j = 0; j < arr.length - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

// Binary search (requires sorted input, O(log n))
public static int binarySearch(int[] arr, int target) {
    int low = 0, high = arr.length - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

```

27.5 Trees

A binary search tree (BST) maintains elements such that every node's left subtree contains smaller values and its right subtree contains larger values, enabling $O(\log n)$ average-case search, insert, and delete on balanced trees.

```

class TreeNode {
    int value;
    TreeNode left, right;
    TreeNode(int value) { this.value = value; }
}

public void insert(TreeNode root, int value) {
    if (value < root.value) {
        if (root.left == null) root.left = new TreeNode(value);
        else insert(root.left, value);
    } else {
        if (root.right == null) root.right = new TreeNode(value);
        else insert(root.right, value);
    }
}

```

```
}  
}
```

Chapter 28: JVM Internals and Class Loading

28.1 The Class Loading Process

Before any class can be used, the JVM must load, link, and initialize it. This process is handled by class loaders, arranged in a hierarchical delegation model.

- Bootstrap ClassLoader — loads core Java classes from the JDK (java.lang, java.util, etc.)
- Platform ClassLoader — loads platform-specific classes
- Application ClassLoader — loads classes from your application's classpath

Loading follows a delegation model: a class loader first asks its parent to load a class before attempting to load it itself, which prevents core classes from being accidentally overridden by application code.

28.2 Linking and Initialization

Linking consists of three phases: verification (ensuring bytecode is structurally valid and doesn't violate security constraints), preparation (allocating memory for static fields with default values), and resolution (converting symbolic references into direct references). Initialization then executes static initializers and assigns declared values to static fields, occurring lazily on first active use of the class.

28.3 Bytecode and the JIT Compiler

Java source compiles to platform-independent bytecode, a compact instruction set understood by the JVM. Initially, the JVM interprets bytecode instruction by instruction. As certain methods are executed frequently ('hot' methods), the Just-In-Time (JIT) compiler compiles them into optimized native machine code, dramatically improving performance for long-running applications.

28.4 Java Platform Module System (JPMS)

Introduced in Java 9, the module system allows applications to be organized into explicit modules with well-defined dependencies and controlled visibility, declared in a module-info.java file at the root of a module.

```
module com.example.orders {
    requires java.sql;
    requires com.example.shared;

    exports com.example.orders.api;
}
```

Modules improve encapsulation beyond package-private visibility by allowing a module to expose only specific packages to specific consumers, and enable smaller, custom runtime images via jlink.

Chapter 29: Security in Java Applications

29.1 Common Vulnerabilities

- SQL Injection — mitigated by always using PreparedStatement with bound parameters instead of string-concatenated SQL.
- Insecure Deserialization — avoid deserializing untrusted data with native Java serialization; prefer validated JSON with allow-listed types.
- Cross-Site Scripting (XSS) — when generating HTML in web applications, always encode user-supplied data before rendering it.
- Hardcoded Credentials — never embed passwords, API keys, or secrets directly in source code; use environment variables or a secrets manager.
- Path Traversal — validate and normalize any file paths derived from user input before using them for file access.

29.2 Cryptography Basics

The `javax.crypto` and `java.security` packages provide APIs for hashing, encryption, and digital signatures. For password storage, never use a fast general-purpose hash like MD5 or SHA-256 alone; use a slow, salted algorithm designed for passwords, such as bcrypt or Argon2 (via a well-maintained library), or PBKDF2 from the standard library.

```
MessageDigest digest = MessageDigest.getInstance("SHA-256");
byte[] hash = digest.digest("some data".getBytes(StandardCharsets.UTF_8));

// AES encryption (simplified example)
KeyGenerator keyGen = KeyGenerator.getInstance("AES");
keyGen.init(256);
SecretKey secretKey = keyGen.generateKey();

Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
cipher.init(Cipher.ENCRYPT_MODE, secretKey);
byte[] encrypted = cipher.doFinal("secret message".getBytes());
```

29.3 Secure Coding Checklist

- Validate and sanitize all external input, including HTTP parameters, file uploads, and message queue payloads.
- Apply the principle of least privilege to database accounts, file permissions, and service credentials.
- Keep dependencies up to date and monitor for known vulnerabilities using tools like OWASP Dependency-Check.
- Use HTTPS/TLS for all network communication carrying sensitive data.
- Log security-relevant events, but never log sensitive data such as passwords or full credit card numbers.

Chapter 30: Performance Tuning and Profiling

30.1 Measure Before You Optimize

Premature optimization without measurement is one of the most common mistakes even experienced developers make. Always profile first to identify actual bottlenecks rather than guessing, since intuition about performance is frequently wrong.

30.2 Profiling Tools

- Java Flight Recorder (JFR) — a low-overhead profiling and event-collection framework built into the JDK, ideal for production diagnostics.
- VisualVM — a free visual tool for monitoring heap, threads, and CPU usage during development.
- JMH (Java Microbenchmark Harness) — the standard tool for writing reliable microbenchmarks that account for JIT warm-up and other JVM quirks.

30.3 Common Performance Pitfalls

- String concatenation inside loops using `+` instead of `StringBuilder`.
- Excessive autoboxing/unboxing between primitives and their wrapper classes in hot loops.
- Using the wrong collection type for the access pattern (e.g. `LinkedList` for random access).
- N+1 query problems when using ORMs like Hibernate, where a loop triggers a separate database query per iteration.
- Unbounded caches that grow indefinitely and eventually cause memory pressure.

30.4 JVM Tuning Flags

```
-Xms512m -Xmx2g           # initial and maximum heap size
-XX:+UseG1GC              # select the G1 garbage collector
-XX:+HeapDumpOnOutOfMemoryError # capture a heap dump on OOM for later analysis
-XX:MaxGCPauseMillis=200   # target maximum GC pause time (G1)
```

Reminder: JVM tuning flags should always be validated with real load testing in a staging environment before being applied in production — defaults are reasonable for most workloads.

Appendix A: Java Keywords Reference

Java reserves the following words, which cannot be used as identifiers (variable, method, or class names).

- abstract, assert, boolean, break, byte, case, catch, char, class, const*
- continue, default, do, double, else, enum, extends, final, finally, float
- for, goto*, if, implements, import, instanceof, int, interface, long, native
- new, package, private, protected, public, return, short, static, strictfp, super
- switch, synchronized, this, throw, throws, transient, try, void, volatile, while

(*const and goto are reserved but not currently used by the language.) Additionally, true, false, and null are reserved literals, and var, yield, and record are contextual keywords usable as identifiers in most contexts.

Appendix B: Quick Reference — Collections Decision Guide

B.1 Choosing a List Implementation

- Need fast random access by index, and mostly append/read? → ArrayList
- Need frequent insertions/removals at the beginning or middle? → LinkedList
- Need thread-safe access from multiple threads? → Collections.synchronizedList(...) or CopyOnWriteArrayList for read-heavy workloads

B.2 Choosing a Set Implementation

- Need fastest average performance, order doesn't matter? → HashSet
- Need insertion order preserved? → LinkedHashSet
- Need elements kept in sorted order? → TreeSet

B.3 Choosing a Map Implementation

- Need fastest average performance, order doesn't matter? → HashMap
- Need insertion or access order preserved? → LinkedHashMap
- Need keys kept in sorted order? → TreeMap
- Need thread safety without external synchronization? → ConcurrentHashMap

B.4 Common Time Complexities

- ArrayList: get $O(1)$, add at end amortized $O(1)$, add/remove at index $O(n)$
- LinkedList: get $O(n)$, add/remove at ends $O(1)$
- HashMap/HashSet: get/put/contains average $O(1)$, worst case $O(n)$
- TreeMap/TreeSet: get/put/contains $O(\log n)$

Chapter 31: Case Study — Building a Library Management System

31.1 Project Overview

This case study ties together everything covered so far — classes, inheritance, interfaces, collections, generics, exceptions, and streams — into a single cohesive console application: a library management system that tracks books, members, and loans.

31.2 Domain Model

We start by modeling the core entities. A `Book` represents an item in the catalog, a `Member` represents a library patron, and a `Loan` represents a book currently checked out.

```
public class Book {
    private final String isbn;
    private final String title;
    private final String author;
    private boolean available = true;

    public Book(String isbn, String title, String author) {
        this.isbn = isbn;
        this.title = title;
        this.author = author;
    }

    public String getIsbn() { return isbn; }
    public String getTitle() { return title; }
    public String getAuthor() { return author; }
    public boolean isAvailable() { return available; }
    public void setAvailable(boolean available) { this.available = available; }

    @Override
    public String toString() {
        return title + " by " + author + " (" + isbn + ") " + (available ? "[Available]" : "[Checked Out]");
    }
}

public class Member {
    private final String memberId;
    private final String name;
    private final List<Loan> loanHistory = new ArrayList<>();

    public Member(String memberId, String name) {
        this.memberId = memberId;
        this.name = name;
    }

    public String getMemberId() { return memberId; }
    public String getName() { return name; }
    public List<Loan> getLoanHistory() { return loanHistory; }
```

```

        public long activeLoanCount() {
            return loanHistory.stream().filter(loan -> !loan.isReturned()).count();
        }
    }

    public class Loan {
        private final Book book;
        private final Member member;
        private final LocalDate dueDate;
        private boolean returned = false;

        public Loan(Book book, Member member, LocalDate dueDate) {
            this.book = book;
            this.member = member;
            this.dueDate = dueDate;
        }

        public Book getBook() { return book; }
        public Member getMember() { return member; }
        public LocalDate getDueDate() { return dueDate; }
        public boolean isReturned() { return returned; }
        public void markReturned() { returned = true; }

        public boolean isOverdue() {
            return !returned && LocalDate.now().isAfter(dueDate);
        }
    }
}

```

31.3 Custom Exceptions

```

public class BookNotAvailableException extends Exception {
    public BookNotAvailableException(String message) { super(message); }
}

public class LoanLimitExceededException extends Exception {
    public LoanLimitExceededException(String message) { super(message); }
}

```

31.4 The Library Service

The Library class encapsulates the business logic, using a Map for O(1) book lookups by ISBN and enforcing rules such as a maximum of three simultaneous loans per member.

```

public class Library {
    private static final int MAX_LOANS_PER_MEMBER = 3;

    private final Map<String, Book> catalog = new HashMap<>();
    private final Map<String, Member> members = new HashMap<>();
    private final List<Loan> activeLoans = new ArrayList<>();

    public void addBook(Book book) {
        catalog.put(book.getIsbn(), book);
    }

    public void registerMember(Member member) {
        members.put(member.getMemberId(), member);
    }
}

```

```

    }

    public Loan checkOut(String isbn, String memberId) throws
BookNotAvailableException, LoanLimitExceededException {
        Book book = catalog.get(isbn);
        Member member = members.get(memberId);

        if (book == null || !book.isAvailable()) {
            throw new BookNotAvailableException("Book with ISBN " + isbn + " is not
available.");
        }
        if (member.activeLoanCount() >= MAX_LOANS_PER_MEMBER) {
            throw new LoanLimitExceededException(member.getName() + " has reached the
loan limit.");
        }

        book.setAvailable(false);
        Loan loan = new Loan(book, member, LocalDate.now().plusWeeks(2));
        member.getLoanHistory().add(loan);
        activeLoans.add(loan);
        return loan;
    }

    public void returnBook(String isbn) {
        activeLoans.stream()
            .filter(loan -> loan.getBook().getIsbn().equals(isbn) &&
!loan.isReturned())
            .findFirst()
            .ifPresent(loan -> {
                loan.markReturned();
                loan.getBook().setAvailable(true);
            });
    }

    public List<Loan> getOverdueLoans() {
        return activeLoans.stream()
            .filter(Loan::isOverdue)
            .collect(Collectors.toList());
    }

    public List<Book> searchByAuthor(String author) {
        return catalog.values().stream()
            .filter(book -> book.getAuthor().equalsIgnoreCase(author))
            .collect(Collectors.toList());
    }
}

```

31.5 Putting It Together

```

public class LibraryApp {
    public static void main(String[] args) {
        Library library = new Library();

        library.addBook(new Book("978-0134685991", "Effective Java", "Joshua
Bloch"));
        library.addBook(new Book("978-0596009205", "Head First Design Patterns",

```

```

"Freeman & Robson"));
    library.registerMember(new Member("M001", "Akhil"));

    try {
        Loan loan = library.checkOut("978-0134685991", "M001");
        System.out.println("Checked out: " + loan.getBook());
        System.out.println("Due date: " + loan.getDueDate());

        library.returnBook("978-0134685991");
        System.out.println("Book returned successfully.");

    } catch (BookNotAvailableException | LoanLimitExceededException e) {
        System.out.println("Checkout failed: " + e.getMessage());
    }
}
}

```

31.6 Extending the Project

This foundation can be extended in many directions that mirror real professional work: persisting the catalog to a database using JDBC or Spring Data JPA instead of in-memory maps, exposing the operations as a REST API with Spring Boot, adding fine and reservation systems, writing a comprehensive JUnit test suite for the Library class, and introducing concurrency controls if multiple librarians could check out books simultaneously.

Exercise: *Extend this project by adding a reservation queue: when a member tries to check out a book that is unavailable, they can join a waiting list and are notified when it becomes available again.*

Chapter 32: Case Study — A Task Management REST API with Spring Boot

32.1 Project Overview

This second case study builds a small but realistic professional-grade REST API for managing tasks, demonstrating how the concepts from earlier chapters — dependency injection, JPA repositories, exception handling, and testing — come together in a modern Spring Boot application.

32.2 The Entity

```
@Entity
@Table(name = "tasks")
public class Task {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String title;

    private String description;

    @Enumerated(EnumType.STRING)
    private TaskStatus status = TaskStatus.PENDING;

    private LocalDate dueDate;

    // constructors, getters, and setters omitted for brevity
}

public enum TaskStatus {
    PENDING, IN_PROGRESS, COMPLETED
}
```

32.3 The Repository

```
public interface TaskRepository extends JpaRepository<Task, Long> {
    List<Task> findByStatus(TaskStatus status);
    List<Task> findByDueDateBefore(LocalDate date);
}
```

32.4 The Service Layer

The service layer holds business logic and stays independent of the web layer, making it straightforward to unit test without spinning up an HTTP server.

```
@Service
public class TaskService {
    private final TaskRepository repository;

    public TaskService(TaskRepository repository) {
```

```

        this.repository = repository;
    }

    public Task createTask(Task task) {
        return repository.save(task);
    }

    public Task getTask(Long id) {
        return repository.findById(id)
            .orElseThrow(() -> new TaskNotFoundException("Task " + id + " not found"));
    }

    public Task updateStatus(Long id, TaskStatus status) {
        Task task = getTask(id);
        task.setStatus(status);
        return repository.save(task);
    }

    public List<Task> getOverdueTasks() {
        return repository.findByDueDateBefore(LocalDate.now());
    }

    public void deleteTask(Long id) {
        repository.deleteById(id);
    }
}

```

32.5 The Controller

```

@RestController
@RequestMapping("/api/tasks")
public class TaskController {
    private final TaskService taskService;

    public TaskController(TaskService taskService) {
        this.taskService = taskService;
    }

    @PostMapping
    public ResponseEntity<Task> create(@RequestBody Task task) {
        Task saved = taskService.createTask(task);
        return ResponseEntity.status(HttpStatus.CREATED).body(saved);
    }

    @GetMapping("/{id}")
    public Task get(@PathVariable Long id) {
        return taskService.getTask(id);
    }

    @PatchMapping("/{id}/status")
    public Task updateStatus(@PathVariable Long id, @RequestParam TaskStatus status)
    {
        return taskService.updateStatus(id, status);
    }
}

```

```

    @GetMapping("/overdue")
    public List<Task> overdue() {
        return taskService.getOverdueTasks();
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        taskService.deleteTask(id);
        return ResponseEntity.noContent().build();
    }
}

```

32.6 Global Exception Handling

Rather than scattering try-catch blocks across every controller method, Spring's `@RestControllerAdvice` provides a centralized place to translate exceptions into consistent HTTP error responses.

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(TaskNotFoundException.class)
    public ResponseEntity<String> handleNotFound(TaskNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(ex.getMessage());
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<String> handleGeneral(Exception ex) {
        return
ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body("Unexpected error
occurred");
    }
}

```

32.7 Testing the Service Layer

```

@ExtendWith(MockitoExtension.class)
class TaskServiceTest {

    @Mock
    private TaskRepository repository;

    @InjectMocks
    private TaskService taskService;

    @Test
    void updatesTaskStatus() {
        Task task = new Task();
        task.setId(1L);
        task.setStatus(TaskStatus.PENDING);

        when(repository.findById(1L)).thenReturn(Optional.of(task));
        when(repository.save(any(Task.class))).thenReturn(task);

        Task updated = taskService.updateStatus(1L, TaskStatus.COMPLETED);
    }
}

```

```

        assertEquals(TaskStatus.COMPLETED, updated.getStatus());
        verify(repository).save(task);
    }

    @Test
    void throwsWhenTaskNotFound() {
        when(repository.findById(99L)).thenReturn(Optional.empty());
        assertThrows(TaskNotFoundException.class, () -> taskService.getTask(99));
    }
}

```

32.8 Deployment Considerations

A production deployment of this API would typically add: input validation with Bean Validation (`@Valid`, `@NotBlank`, `@Future`), pagination on list endpoints, authentication and authorization via Spring Security, API documentation via OpenAPI/Swagger, database migrations managed by Flyway or Liquibase, and containerization with Docker for consistent deployment across environments.

Exercise: Add a *User* entity with a one-to-many relationship to *Task*, and secure the API so users can only view and modify their own tasks.

Appendix C: Glossary of Java Terms

Core Language

- Bytecode — the platform-independent instruction set produced by the Java compiler and executed by the JVM.
- Class — a blueprint defining the fields and methods that its instances (objects) will have.
- Compilation Unit — a single .java source file.
- Constructor — a special method used to initialize a newly created object.
- Encapsulation — bundling data and the methods that operate on it, restricting direct access to internal state.
- Immutable — an object whose state cannot change after construction, such as String.
- Inheritance — a mechanism where a class acquires fields and methods from a parent class.
- Interface — a contract specifying method signatures that implementing classes must provide.
- Overloading — defining multiple methods with the same name but different parameter lists.
- Overriding — providing a new implementation of an inherited method in a subclass.
- Polymorphism — the ability of a reference to behave differently depending on the actual object type at runtime.

Runtime and Memory

- Garbage Collection — the automatic process of reclaiming memory occupied by unreachable objects.
- Heap — the runtime memory region where objects are allocated.
- JIT (Just-In-Time) Compiler — converts frequently executed bytecode into optimized native machine code at runtime.
- JVM (Java Virtual Machine) — the runtime engine that executes Java bytecode.
- Stack — per-thread memory holding local variables and method call frames.

Collections and Generics

- Collection — a general-purpose container for grouping multiple elements.
- Generics — a language feature enabling classes and methods to operate on parameterized types with compile-time safety.
- Iterator — an object that enables sequential traversal of a collection.
- Wildcard — the ? symbol used in generics to represent an unknown type, often bounded with extends or super.

Concurrency

- Deadlock — a state where two or more threads are blocked forever, each waiting on the other.
- Executor — an abstraction for managing and reusing a pool of threads to execute submitted tasks.

- Race Condition — a bug where the outcome depends on the unpredictable timing of concurrent thread execution.
- Thread — the smallest unit of independently scheduled execution within a process.

Modern Java

- Lambda Expression — a concise, anonymous implementation of a functional interface.
- Optional — a container object used to represent the possible absence of a value, reducing null-related bugs.
- Record — a compact, immutable data-carrier class with automatically generated accessors and equality methods.
- Stream — a sequence of elements supporting functional-style operations such as filter, map, and reduce.

Ecosystem and Tooling

- Dependency Injection — a pattern where an object's dependencies are provided externally rather than created internally.
- JAR (Java Archive) — a packaged, compressed collection of compiled classes and resources.
- Maven / Gradle — build automation and dependency management tools widely used in Java projects.
- ORM (Object-Relational Mapping) — a technique for mapping objects to relational database tables, e.g. via Hibernate.
- Spring Boot — a framework built on Spring that simplifies configuration and accelerates building production-ready applications.